

Explicando Decisões de LLMs via Otimização Inversa

George Gomes

Programa de Pós-Graduação

2026

Motivação — Por que investigar decisões de LLMs?

- LLMs são “**caixas-pretas**”: tomam decisões mas não explicam o critério
- Pergunta central:
*“LLMs possuem um processo de decisão **estável** e **aprendível**?”*
- Se sim $\Rightarrow$ podemos extrair uma **métrica matemática** ($\hat{W}$) que **explique** e **preveja** suas decisões
- Relevância:
 - Explainability / Interpretabilidade
 - Confiança em sistemas de IA
 - Auditoria de decisões automatizadas

Hipóteses do Trabalho

- H1 Consistência:** O LLM mantém o mesmo critério decisório em problemas diferentes
- H2 Few-shot amplifica:** Exemplos rotulados aumentam a concordância LLM–métrica
- H3 LLM como aprendiz:** O LLM consegue aprender o critério de um perito externo
- H4 Exemplos difíceis:** Exemplos próximos à fronteira são mais informativos
- H5 Estabilidade:** O comportamento é reproduzível entre execuções

Abordagem — Otimização Inversa

Ideia Geral

A partir das **decisões observadas** do LLM, aprender a **função objetivo implícita**.

Métrica de Mahalanobis diagonal:

$$d_W(x, c) = \sqrt{\sum_i w_i \cdot (x_i - c_i)^2}, \quad w_i \geq 0$$

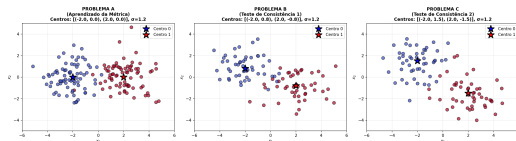
Por que diagonal?

- Simplicidade: $O(d)$ parâmetros vs. $O(d^2)$ na Mahalanobis completa
- Interpretabilidade: cada w_i indica a importância da feature i
- Tratabilidade computacional

Classificação: nearest-centroid — atribuir ao centróide mais próximo sob W .

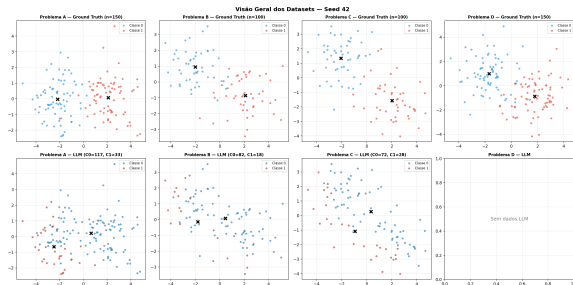
Dados Sintéticos 2D — Visão Geral

- 4 problemas gerados com `make_blobs`
- Distribuições Gaussianas bidimensionais
- Dados 2D permitem **visualização direta** das fronteiras de decisão
- Cada problema tem geometria distinta



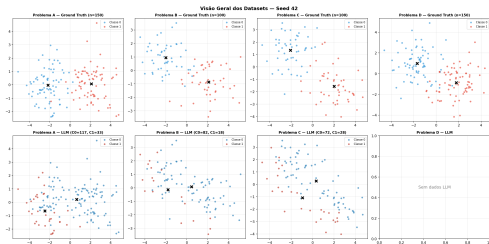
Problema A — Treino da Métrica

- 150 pontos, 2 classes
- Usado na **Fase A**: LLM classifica em zero-shot
- A partir das decisões, aprende-se $\hat{W}_{\text{LLM}}$
- Parâmetros: centroides e `cluster_std` do `make_blobs`



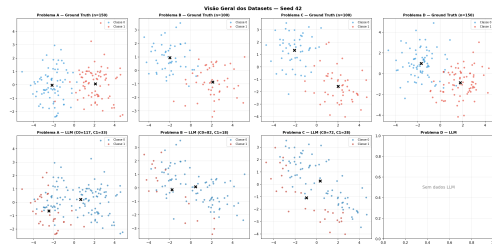
Problema B — Teste de Consistência 1

- 100 pontos, centroides deslocados verticalmente
- Geometria **diferente** do Problema A
- Testa se $\hat{W}_{LLM}$ aprendida na Fase A **generaliza**
- Mesmo número de classes, distribuição distinta



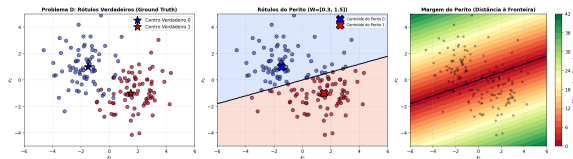
Problema C — Teste de Consistência 2

- 100 pontos, distorção geométrica maior
- Teste mais **desafiador** que o Problema B
- Validação adicional da generalização da métrica
- Se a consistência se mantém $\Rightarrow$ evidência forte de estabilidade



Problema D — Expert Externo

- 150 pontos, rotulado por expert com W_{expert} conhecido
- $W_{\text{expert}} = [0.3, 1.5]$: x_2 dominante ($5\times$)
- Fronteira de decisão **não-trivial**
- Papel invertido: LLM **aprende** do expert



Configurações de Execução — Design de Variabilidade

Objetivo

Garantir que resultados não são artefato de uma configuração específica.

Parâmetros de robustez:

- Seeds: {42, 123, 7} (3 sementes)
- Repetições: 3 por configuração
- Temperatura: 0.0 (minimiza estocasticidade)
- Modelo: gpt-4o-mini

Variações de nomes de classes:

- A/B — neutro
- 0/1 — numérico
- Positivo/Negativo — semântico
- Azul/Vermelho — semântico

⇒ Detecta viés semântico do LLM

Algoritmo 1 — Perceptron Estruturado com Relaxação de Margem

Ideia

Busca iterativa dos pesos W que maximizam a margem de separação.

Procedimento:

- 1 **Busca binária** sobre γ (margem mínima)
- 2 Para cada γ , treina o perceptron até convergência ou limite de épocas
- 3 **Atualização:** quando a métrica erra a classificação do LLM:

$$w_i \leftarrow w_i + \eta \cdot [(x_i - c_{\text{errado}})^2 - (x_i - c_{\text{correto}})^2]$$

- 4 **Projeção:** $w_i \leftarrow \max(w_i, 0)$ (não-negatividade)
- 5 Retorna o W com maior γ que converge

Algoritmo 2 — Mínimos Quadrados (NNLS)

Formulação

$$\min_{w \geq 0} \|Aw - b\|^2$$

Construção do sistema:

- Para cada ponto x classificado como classe k pelo LLM:

$$\sum_i w_i (x_i - c_{k,i})^2 \leq \sum_i w_i (x_i - c_{j,i})^2, \quad \forall j \neq k$$

- Reescrito como $Aw \leq b$ e resolvido via NNLS (`scipy`)

Vantagens:

- Solução fechada (não iterativa)
- Garante $w_i \geq 0$ por construção
- Validação cruzada: se Perceptron e NNLS convergem para W similar $\Rightarrow$ resultado robusto

Algoritmo 3 — Programação Linear (Max-Margin)

Formulação

$$\begin{aligned} \max_{w, \gamma} \quad & \gamma \quad \text{s.a.} \quad \sum_i w_i [(x_i - c_{j,i})^2 - (x_i - c_{k,i})^2] \geq \gamma, \quad \forall (x, k), j \neq k \\ & \sum_i w_i = 1, \quad w_i \geq 0, \quad \gamma \geq 0 \end{aligned}$$

Características:

- Maximiza a margem mínima sobre todas as observações
- Solução via `scipy.optimize.linprog` (solver HiGHS)
- Normalização $\sum w_i = 1$ evita solução trivial
- Se $\gamma^* = 0$: separação perfeita impossível com métrica diagonal

Interpretação: $\gamma^* > 0$ indica que existe uma métrica diagonal que reproduz **todas** as decisões do LLM com margem positiva.

Fase A — Aprendizado da Métrica (Zero-Shot)

Procedimento

- 1 LLM classifica **150 pontos** do Problema A em **zero-shot**
- 2 Prompt contém apenas as coordenadas e pede classificação A ou B
- 3 A partir das 150 decisões, aprende-se $\hat{W}_{\text{LLM}}$ via cada algoritmo
- 4 $\hat{W}$ representa o **critério decisório implícito** do LLM

Nota: $\hat{W}_{\text{LLM}}$ é **estimada/inferida**, não “aprendida” pelo LLM — é o resultado da otimização inversa sobre as decisões observadas.

Fase A — $\hat{W}$ Estimado e Fidelidade

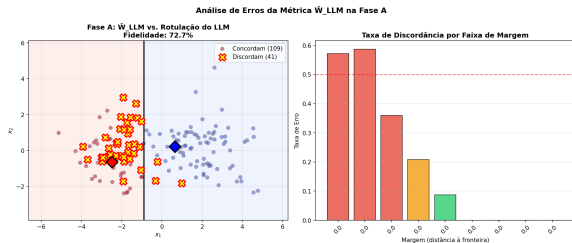
$\hat{W}$ por algoritmo (seed=42, classes A/B):

Algoritmo	$\hat{w}_1$	$\hat{w}_2$
Perceptron	1.000	0.000
NNLS	0.202	0.798
LP	0.707	0.707

Fidelidade (métrica reproduz LLM):

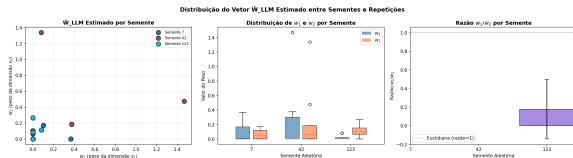
- Perceptron: **72.7%**
- NNLS: **80.7%**
- LP: **74.7%**

Acurácia LLM vs. ground truth: apenas 30.7% — mas a fidelidade mede se a métrica reproduz o LLM, não se o LLM acerta.



Fase A — Distribuição de $\hat{W}$ entre Seeds

- $\hat{W}$ varia entre seeds
- Similaridade cosseno entre seeds: **0.53** (média)
- Seed 42: $\hat{W} = [1.0, 0.0]$ (Perceptron)
- Seeds 123, 7: $\hat{W} = [0.0, 1.0]$
- **H5 em risco**: direção de W instável
- CV da razão w_1/w_2 : **258.9%**



Fase A — Comparação de Algoritmos

Concordância entre algoritmos:

- Cosseno Perc–NNLS: **0.73**
- Cosseno Perc–LP: **0.73**
- Cosseno NNLS–LP: **0.87**

Fidelidade média (3 seeds):

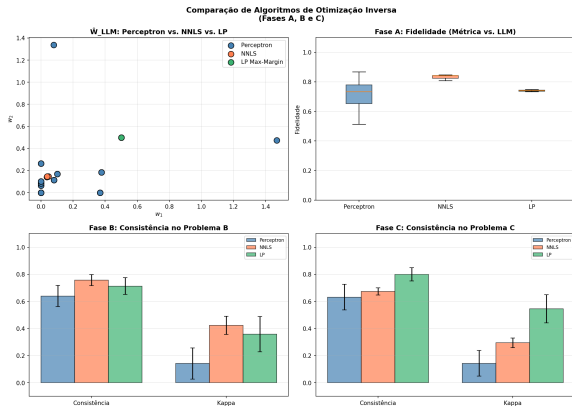
Algoritmo	Fidelidade
-----------	------------

Perceptron	76.0%
------------	-------

NNLS	82.4%
------	-------

LP	75.2%
----	-------

NNLS e LP mais concordantes entre si;
Perceptron mais divergente em seeds
específicas.



Fases B e C — Teste de Consistência

Pergunta

“A métrica aprendida na Fase A **generaliza** para problemas diferentes?”

Procedimento:

- 1 Aplica $\hat{W}_{\text{LLM}}$ (Fase A) ao Problema B/C com centroides locais
- 2 LLM classifica os mesmos pontos (zero-shot ou few-shot)
- 3 Compara: classificação do LLM vs. predição da métrica

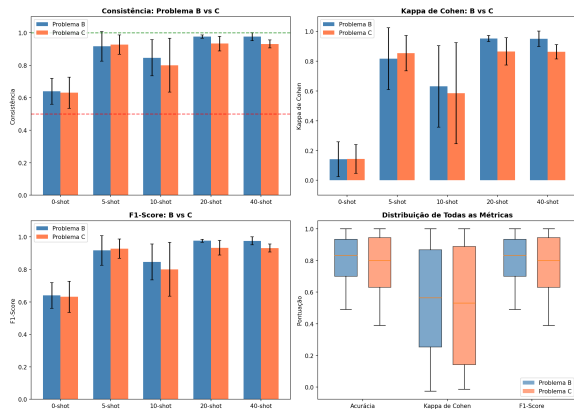
Métricas:

- **Consistência (%)**: fração de concordância LLM–métrica
- **Cohen's Kappa (κ)**: concordância ajustada pelo acaso
- **F1-Score**: média harmônica de precisão e recall

Fases B e C — Resultados (Perceptron, classes A/B)

n_shot	Prob. B		Prob. C	
	Cons.	κ	Cons.	κ
0	64.0%	0.14	63.1%	0.14
5	91.7%	0.82	92.7%	0.85
10	96.3%	0.93	96.3%	0.93
20	97.6%	0.95	93.3%	0.87
40	97.6%	0.95	93.1%	0.86

Salto dramático de zero-shot para 5-shot:
+27.7% (B), +29.6% (C)



Fases B e C — Discussão

H1 — Consistência

- Zero-shot: $\kappa \approx 0.14 \Rightarrow$ **concordância fraca**
- Few-shot (≥ 5): $\kappa > 0.80 \Rightarrow$ **concordância substancial**
- **H1 parcialmente confirmada:** consistência **com** exemplos, não sem

H2 — Few-shot amplifica

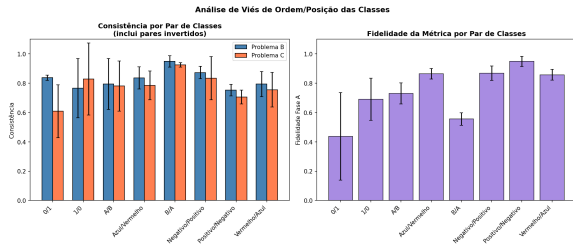
- Exemplos rotulados pela métrica **aumentam drasticamente** a concordância
- 5 exemplos já bastam para $\kappa > 0.80$
- **H2 confirmada**

Limitação: fidelidade baixa na Fase A (72–81%) $\Rightarrow$ base frágil para transferência em zero-shot.

Viés de Ordem das Classes

Pergunta: Trocar “A ou B” por “B ou A” muda o resultado?

Classes	B (%)	C (%)
A/B	79.5	78.1
B/A	94.9	92.5
0/1	83.7	60.9
1/0	76.7	82.8
Pos./Neg.	75.3	70.5
Neg./Pos.	87.3	83.5
Azul/Verm.	83.6	78.5
Verm./Azul	79.4	75.6



Existe viés posicional: inverter a ordem pode alterar consistência em até **+15.4%** (B/A vs A/B).

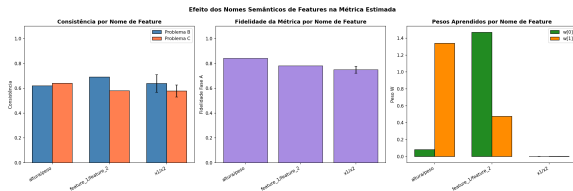
Nomes Semânticos de Features

Pergunta: Chamar as coordenadas de “altura/peso” muda $\hat{W}$?

Variantes testadas:

- x1/x2 — neutro (padrão)
- altura/peso — semântico
- feature_1/feature_2 — técnico

O LLM é **sensível** ao significado semântico das variáveis — nomes como “altura/peso” podem ativar conhecimento prévio e alterar pesos implícitos.



Variantes de Prompt (1/2) — Design

Pergunta: Mudar o template do prompt muda a métrica $\hat{W}$ inferida?

4 variantes testadas:

`default` Template atual (baseline)

`geometric` Adiciona contexto geométrico/espacial explícito

`chain-of-thought` Pede raciocínio passo a passo antes da resposta

`tabular` Apresenta coordenadas como tabela markdown

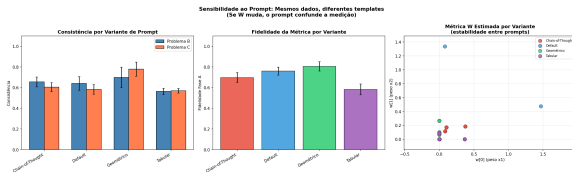
Protocolo:

- Cada variante executa Fases A–C completas (zero-shot, classes A/B) em todas as seeds
- Compara $\hat{W}$ aprendido, consistência e κ

Variantes de Prompt (2/2) — Resultados

- O scatter de $\hat{W}$ mostra se o prompt “confunde” a medição
- Se $\hat{W}$ muda entre variantes $\Rightarrow$ o prompt é um confundidor
- **Implicação:** a métrica inferida pode não capturar algo intrínseco ao LLM, mas sim um artefato do prompt

Resultado: sensibilidade moderada — variantes com contexto explícito tendem a alterar a distribuição de decisões.

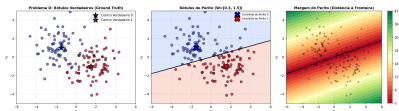


Fase D — LLM como Aprendiz

Papel Invertido

Agora o LLM **aprende** de um expert externo com métrica W_{expert} conhecida.

- 1 Expert gera rótulos no Problema D usando $W_{\text{expert}} = [0.3, 1.5]$
- 2 LLM recebe exemplos few-shot rotulados pelo expert
- 3 LLM classifica pontos novos
- 4 Pergunta: “O LLM consegue aprender o critério do expert?”



Estratégias de Seleção de Exemplos

4 estratégias:

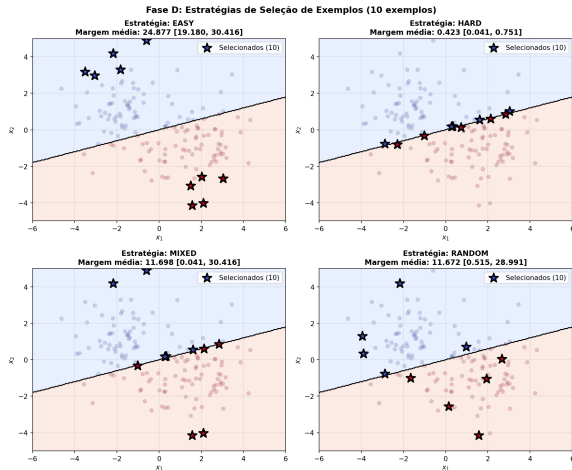
Easy Pontos longe da fronteira (alta margem)

Hard Pontos próximos à fronteira (ambíguos)

Mixed 50% easy + 50% hard

Random Baseline sem estratégia

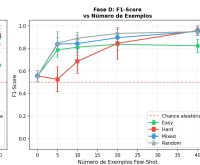
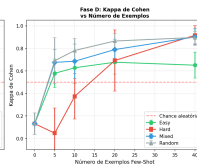
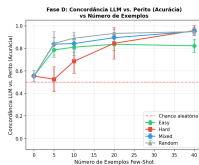
Intuição H4: exemplos hard deveriam ser mais informativos por definirem melhor a fronteira.



Fase D — Curvas de Aprendizado (Expert aniso_x2)

n	Easy	Hard	Mixed	Random
0	55.5	55.5	55.5	55.5
5	78.6	52.5	83.7	84.4
10	81.1	68.5	84.3	89.0
20	83.6	84.5	89.5	93.2
40	82.3	95.8	95.1	94.8

H3 confirmada: LLM melhora de 55.5% (zero-shot) para até **95.8%** (40-shot hard).



Fase D — Comparação de Estratégias

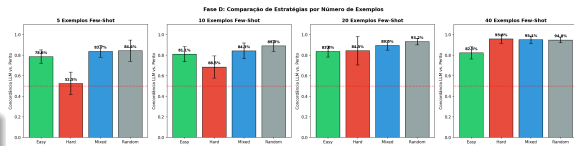
Acurácia média (todos $n_shot > 0$):

- 1 Random: 90.4%
- 2 Mixed: 88.1%
- 3 Easy: 81.4%
- 4 Hard: 75.3%

H4 REFUTADA

Exemplos easy (81.4%) superam hard (75.3%) consistentemente.

Interpretação: exemplos “hard” são **ambíguos** para o LLM. Exemplos fáceis funcionam como “âncoras” que definem regiões de cada classe.



Múltiplos Experts

3 configurações de expert testadas:

Expert	W_{expert}	Razão	Descrição
aniso_x2	[0.3, 1.5]	$5.0\times$	x_2 dominante (original)
aniso_x1	[1.5, 0.3]	$0.2\times$	x_1 dominante (invertido)
euclidean	[1.0, 1.0]	$1.0\times$	Pesos iguais (Euclidiana)

Pergunta: “O LLM aprende igualmente bem critérios diferentes?”

Validação oráculo (oracle): todos os 3 algoritmos recuperam W_{expert} com **fidelidade 100%** no Problema D $\Rightarrow$ os algoritmos funcionam corretamente quando o classificador é consistente.

Similaridade cosseno (recuperado vs. real): **0.99–1.00** para Perceptron e LP.

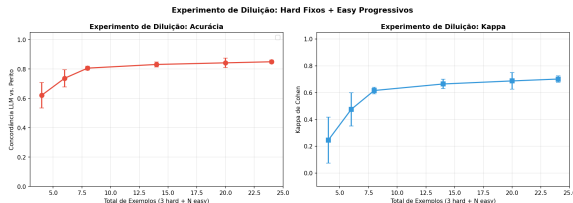
Experimento de Diluição

Pergunta: Adicionar exemplos fáceis
“diluí” a informação dos difíceis?

Design: 4 hard fixos + easy progressivos:

Hard	Easy	Acc (%)
4	0	56.8
4	2	65.9
4	4	81.7
4	10	83.1
4	16	86.9
4	20	84.9

Fáceis **não diluem** — performance
melhora monotonicamente até 16 easy, com
leve queda em 20.



Viés de Ordem dos Exemplos Few-Shot (Recency Bias)

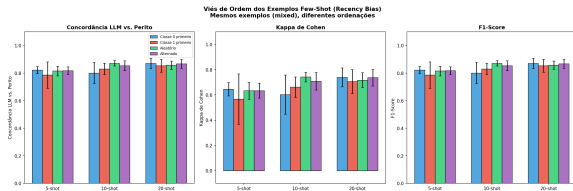
Pergunta: A ordem dos exemplos few-shot afeta a performance?

4 ordenações (mesmos exemplos, estratégia mixed):

- class0_first: classe 0 primeiro
- class1_first: classe 1 primeiro
- shuffled: ordem aleatória
- alternating: intercalado

Resultados (seed=42, 10-shot):

Ordem	Acc (%)
class0_first	90.0
class1_first	88.6
shuffled	89.0
alternating	89.0



Validação Oráculo (Sanity Check)

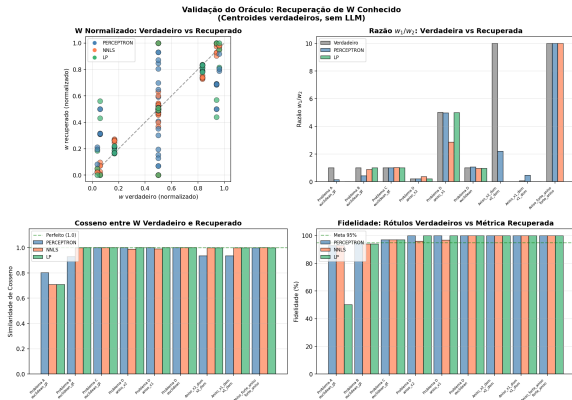
Objetivo: verificar se os algoritmos **funcionam** quando o classificador é consistente.

Protocolo:

- 1 Classificador sintético com W conhecido gera rótulos
- 2 Os 3 algoritmos tentam recuperar W

Resultados (Prob. D, aniso_x2):

Alg.	Cos. sim.	Fidelidade
Perceptron	0.9999	100%
NNLS	0.9933	95.3%
LP	0.9999	100%



Se não recuperam $\Rightarrow$ problema é o algoritmo. Se recuperam $\Rightarrow$ problema é a inconsistência do LLM.

Projeção R3 — Kernel Quadrático (Metodologia)

Motivação: testar se o LLM captura relações não-lineares entre features.

Como funciona:

- Dados 2D originais $(x_1, x_2) \Rightarrow$ adiciona $x_3 = x_1 \cdot x_2$
- Simula kernel quadrático sem sair do mundo linear
- LLM classifica 150 pontos com 3 coordenadas no prompt
- 3 algoritmos aprendem W com 3 componentes

Comparação:

- LLM com 2 features (R2) vs. LLM com 3 features (R3)
- Métrica aprendida em R3

Pergunta: o peso w_3 (associado a $x_1 \cdot x_2$) é relevante? O LLM “percebe” a interação?

Projeção R3 — Resultados (seed=42)

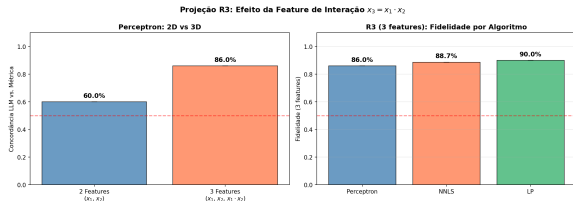
$\hat{W}$ com 3 features:

Alg.	w_1	w_2	w_3
Perc.	2.31	1.21	1.07
NNLS	0.05	0.28	0.03
LP	0.33	0.33	0.33

Fidelidade R3:

- Perceptron: 86.0%
- NNLS: 88.7%
- LP: 90.0%

Concordância LLM(2feat) vs Métrica(3feat): apenas 60.0% $\Rightarrow$ adicionar x_3 muda o comportamento do LLM.



Baselines Clássicos — Configuração

Pergunta: “O LLM faz algo que um classificador trivial não faria?”

Mesmos exemplos few-shot da Fase D $\rightarrow$ treino. **Mesmo** conjunto de teste.

Classificador	Configuração
k-NN	$k = \min(n_{\text{shot}}, 5)$, ímpar. Distância euclidiana.
Logistic Regression	Regularização L2, <code>max_iter=1000</code> . Fronteira linear.
SVM (RBF)	Kernel radial, $C = 1.0$, $\gamma = \text{scale}$. Fronteira não-linear.

Avaliação: mesmas métricas do LLM (acurácia vs. expert, κ , F1). Testado para cada (n_{shot} , estratégia, expert, seed).

Baselines Clássicos — Resultados

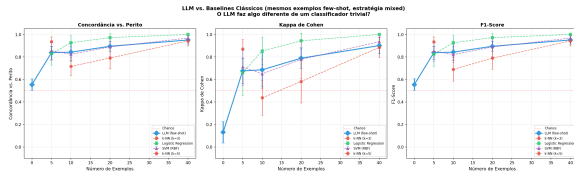
Acc vs. Expert (10-shot, easy, seed=42):

Classificador	Acc (%)
k-NN (k=5)	89.3
Log. Regression	89.3
SVM (RBF)	90.0
LLM	88.6

Com exemplos hard (10-shot):

k-NN	62.1%
Log. Reg.	100.0%
SVM (RBF)	48.6%
LLM	68.5%

Conclusão: LLM não é uniformemente superior. LR se destaca com exemplos hard (fronteira linear ideal). LLM tem vantagem em poucos exemplos ambíguos vs. k-NN/SVM.

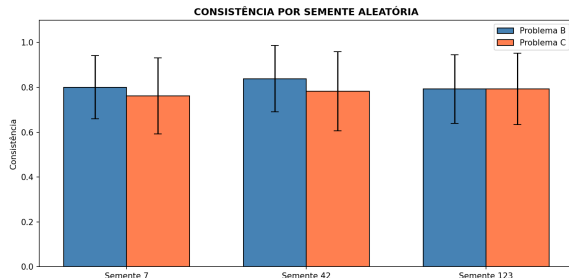


Robustez entre Seeds

Variabilidade dos resultados:

- Consistência B: $81.1\% \pm 14.8\%$
- Consistência C: $77.9\% \pm 16.8\%$
- $\hat{W}$ instável entre seeds (cosseno 0.53)
- Resultados few-shot mais estáveis que zero-shot

Observação: a alta variância em zero-shot reflete a **instabilidade intrínseca** do critério do LLM sem orientação externa.



Testes Estatísticos

Bootstrap 95% CI (10.000 reamostras):

Métrica	n_shot	Média	95% CI
Consistência B (Perc)	0	0.641	[0.602, 0.675]
Consistência B (Perc)	10	0.963	[0.941, 0.985]
Consistência C (Perc)	0	0.584	[0.556, 0.609]
Consistência C (Perc)	10	0.963	[0.956, 0.970]

Teste pareado (zero-shot vs. 10-shot):

- Consistência B: $\Delta = +0.322$, $CI_{95\%} = [+0.257, +0.360]$ **significativo**
- Consistência C: $\Delta = +0.373$, $CI_{95\%} = [+0.342, +0.400]$ **significativo**
- Cohen's d : 7.08 (B), 19.81 (C) $\Rightarrow$ **efeito muito grande**

Easy vs. Hard (Fase D, 5-shot): $\Delta = +0.231$, $CI_{95\%} = [+0.100, +0.340]$ **sig.**

Síntese dos Resultados — Hipóteses

	Hipótese	Resultado	Evidência
H1	Consistência	Parcial	$\kappa = 0.14$ (zero) $\rightarrow 0.93$ (10-shot)
H2	Few-shot amplifica	Confirmada	Salto de +30% com 5 exemplos
H3	LLM como aprendiz	Confirmada	55% $\rightarrow$ 96% (40-shot)
H4	Exemplos difíceis	Refutada	Easy 81% > Hard 75%
H5	Estabilidade	Parcial	$\hat{W}$ instável (cos=0.53)

Achados principais:

- LLMs **não** têm critério estável em zero-shot, mas **respondem muito bem** a exemplos
- Exemplos fáceis (“âncoras”) são mais eficazes que exemplos difíceis (“ambíguos”)
- A refutação de H4 é um **achado relevante** sobre in-context learning

- ① **Framework de otimização inversa aplicado a LLMs**
 - Abordagem nova: inferir critério decisório via Mahalanobis diagonal
 - 3 algoritmos independentes validados com oráculo
- ② **Evidência empírica sobre consistência decisória**
 - LLMs são consistentes **com** orientação (few-shot), não sem
- ③ **Refutação de H4 — achado valioso**
 - LLMs se beneficiam mais de exemplos prototípicos que fronteirços
- ④ **Análise de sensibilidade abrangente**
 - Viés de classe, features semânticas, prompt, ordem de exemplos
 - Identificação de confundidores na medição

Limitações e Trabalhos Futuros

Limitações:

- Apenas um modelo (gpt-4o-mini)
- Dados sintéticos 2D apenas
- Métrica diagonal ($O(d)$ params)
- Temperatura fixa em 0.0
- LP infeasible em vários cenários

Trabalhos futuros:

- Múltiplos LLMs (Claude, Gemini, GPT-4o)
- Dados reais e alta dimensão (Iris, MNIST)
- Mahalanobis completa
- Fronteiras não-lineares (kernels, redes)
- Aplicações: auditoria de decisões de IA
- Variação de temperatura

Obrigado!

Perguntas?

George Gomes
Programa de Pós-Graduação
2026